

GPAS: OPTIMIZER-AWARE TASK ALLOCATION FOR MULTI-TEACHER ON-POLICY DISTILLATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-teacher on-policy distillation (MOPD) uses several specialist teachers to train one student. A task mixture often determines both each task’s weight in the training objective and the number of training examples allocated to that task. We separate these choices. The objective weights remain fixed, every teacher contributes to every optimizer step, and the allocation variable is the number of fixed-size micro-batches assigned to each task. Each micro-batch loss is averaged internally and scaled by its task weight divided by its task’s micro-batch count. The resulting expected gradient is independent of the allocation. GPAS is Neyman allocation in optimizer-scaled coordinates: it assigns micro-batches according to each task’s contribution to the variance of the preconditioned gradient estimate. Cost-GPAS minimizes the product of this variance and a measured step-time model, including allocation-independent overhead. This analysis requires the optimizer scaling to have an allocation-independent target. We meet that requirement with a taskwise AdamW second-moment observation whose expectation depends on the student state but not the micro-batch counts. We specify controlled checks and a four-teacher MOPD comparison that holds prompt streams, total micro-batches, and objective weights fixed while changing only the allocation.

1 INTRODUCTION

Multi-teacher on-policy distillation (MOPD) uses specialist teachers to train a single student on several domains (Ma et al., 2026). A task mixture usually specifies both the learning objective and the share of training compute. An adaptive schedule can therefore change what the student learns while changing the estimator variance. Differences in teacher-loss scale, response length, and serving time make the two effects difficult to separate (Gao et al., 2026; Sun et al., 2026).

We study the setting in which every teacher can contribute to every optimizer step. A step contains G fixed-size micro-batches, and task i receives m_i of them. The target weights w_i are chosen before training and do not change. Scaling each micro-batch loss by w_i/m_i makes the accumulated gradient an estimate of the same weighted objective for every allocation. This differs from averaging all tokens in a mixed step, which makes a task’s effective weight increase with its assigned micro-batch count.

Our method, Gradient-Preconditioned Adaptive Sampling (GPAS), uses the within-task variance of micro-batch gradients after the diagonal scaling given by the current optimizer. It allocates more micro-batches to tasks with larger scaled sampling noise. This is Neyman allocation in optimizer-scaled coordinates. Cost-GPAS also accounts for measured critical-path time and the allocation-independent part of each step. We solve its bounded four-task allocation numerically. A minimum of two micro-batches per task supplies a new noise estimate in every step.

Adaptive micro-batch counts also change the squared accumulated gradient used by conventional AdamW. The allocation derivation treats the diagonal optimizer scaling as fixed. If its second-moment target changes with the counts, this condition fails across steps. We therefore square each unweighted micro-batch gradient before applying w_i/m_i . The resulting taskwise AdamW target depends on the student parameters but not the allocation. Conventional AdamW remains the standard-practice reference.

The paper makes two contributions:

- We define the optimizer-state condition needed to compare allocations, satisfy it with a taskwise AdamW observation, and derive variance-only and time-aware count rules for a fixed MOPD objective. The time-aware objective includes the fixed cost of an optimizer step and teacher visits.
- We give a measurement design that tests gradient variance at frozen checkpoints and training progress under matched prompt streams. It uses optimizer steps for the GPAS comparison and GPU hours for Cost-GPAS.

2 RELATED WORK

Task weighting and data allocation. Multi-task methods tune loss weights, alter gradient directions, or adapt data schedules (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018). Carefully tuned scalarization is competitive with specialized multi-task methods in several settings (Kurin et al., 2022; Xin et al., 2022). Language-model training also adapts domain, rollout, or prompt distributions using learning progress (Albalak et al., 2023; Wang et al., 2025; Zheng et al., 2025; Gao et al., 2025). GPAS keeps task importance fixed and changes the number of micro-batches used to estimate each task gradient.

Online importance samplers observe the loss or gradient of selected items and therefore require bandit feedback for unselected items. The all-task estimator studied here observes every task at every optimizer step.

Multi-teacher on-policy distillation. MOPD routes student rollouts to task teachers and trains a shared student with token-level teacher feedback (Ma et al., 2026). Open-MOPD studies unequal token budgets, reward scales, and stale rewards (Gao et al., 2026). D³-MOPD updates the domain mixture using teacher-loss gap and descent velocity (Sun et al., 2026); this changes the task weights represented by the training mixture. GPAS instead holds w_i fixed and changes m_i , which changes estimator variance but not the expected weighted gradient. Teacher mixtures can also create transfer and task trade-offs (Li et al., 2026). We therefore report task-level capability as well as the fixed teacher-loss objective.

Stratified allocation for optimization. Gradient-norm sampling reduces the second moment of an SGD estimate (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Neyman allocation assigns samples according to within-stratum standard deviation, and unequal sampling costs give the same square-root cost correction used here (Neyman, 1934; Cochran, 1977; Mohri et al., 2026). The appropriate variance scale depends on the optimizer: Euclidean coordinates are natural for SGD, while adaptive optimizers rescale coordinates (Lahire, 2023). GPAS applies this stratified allocation to fixed-size MOPD micro-batches, uses the current AdamW scaling to measure noise, and specifies an allocation-independent AdamW second-moment observation.

3 A FIXED OBJECTIVE AND MICRO-BATCH ESTIMATOR

Assume M teacher tasks. Let $L_i(\theta)$ be task i ’s reverse-KL loss, averaged over valid tokens within a response, over the R responses for a prompt, and over prompts. Positive weights w_i sum to one and define the training objective

$$F(\theta) = \sum_{i=1}^M w_i L_i(\theta). \quad (1)$$

The weights are fixed before training. We use $w_i = 1/M$ when the task losses are normalized as $L_i = \ell_i/\ell_i(0)$. In the experiments we instead use the unnormalized losses and set

$$w_i = \frac{1/\ell_i(0)}{\sum_{j=1}^M 1/\ell_j(0)}. \quad (2)$$

All four initial losses are positive, so no denominator floor is active.

A micro-batch contains c prompt groups, with one prompt and R sampled responses in each group. Its size and hierarchical loss normalization are the same for every task and every allocation rule.

Each optimizer step processes G micro-batches. Task i receives the integer count m_i , where

$$\sum_{i=1}^M m_i = G, \quad m_{\min} \leq m_i \leq m_{\max}. \quad (3)$$

We use $m_{\min} = 2$ so that every task supplies a within-step noise estimate. Thus the main method requires $G \geq 2M$ and includes every teacher in every step.

Let $g_{i,s}$ be the gradient of the internally averaged loss of micro-batch s from task i , and let $a_i = m_i^{-1} \sum_{s=1}^{m_i} g_{i,s}$. We accumulate

$$A = \sum_{i=1}^M w_i a_i = \sum_{i=1}^M \frac{w_i}{m_i} \sum_{s=1}^{m_i} g_{i,s}. \quad (4)$$

In an automatic-differentiation implementation, this is obtained by multiplying each internally averaged micro-batch loss by w_i/m_i before backward. Conditional on the student state and the chosen counts, $\mathbb{E}[A] = \nabla F(\theta)$. Changing m_i therefore changes sampling variance but not the training objective. Averaging all tokens in a step without this factor would instead give task weight approximately m_i/G . In particular, a task at $m_{\min}$ has exactly the same expected gradient contribution as it does under Uniform; its contribution only has higher variance. The count floor therefore preserves task coverage by construction, unlike methods that change mixture weights according to loss or progress.

Taskwise AdamW second moment. Conventional AdamW uses A^2 in its second-moment average. Its expectation changes with the allocation because the noise contribution from task i is proportional to w_i^2/m_i . We instead use the element-wise observation

$$U = \sum_{i=1}^M \frac{w_i}{m_i} \sum_{s=1}^{m_i} g_{i,s}^2. \quad (5)$$

For fixed micro-batch size and student state,

$$\mathbb{E}[U] = \sum_{i=1}^M w_i \mathbb{E}[g_i^2], \quad (6)$$

which depends on θ but not on m . Equation 8 treats D as fixed when optimizing the allocation. Conventional AdamW violates this premise over multiple steps because the target of its second-moment state contains the allocation-dependent noise term. We update AdamW’s first moment with A and its second moment with U ; we call this update *taskwise AdamW*. It makes the target that defines D independent of the allocation and is therefore part of the method, rather than a device for comparing runs. It needs one additional parameter-sized accumulator for U . The task mean a_i needs a second temporary parameter-sized buffer that is reused across tasks. Bias correction, the learning-rate schedule, clipping, and decoupled weight decay remain unchanged.

4 GPAS MICRO-BATCH ALLOCATION

4.1 OPTIMIZER-SCALED WITHIN-TASK NOISE

Let $D = (\sqrt{v} + \epsilon)^{-1}$ be the diagonal scaling obtained from the current taskwise AdamW second-moment state, before the present step. For SGD, D is the identity. Define the optimizer-scaled micro-batch noise of task i as

$$e_i = \mathbb{E} \|D(g_i - \mathbb{E}[g_i])\|_2^2. \quad (7)$$

Assuming independent micro-batches, the scaled variance of Equation 4 is

$$V(m) = \mathbb{E} \|D(A - \nabla F)\|_2^2 = \sum_{i=1}^M \frac{w_i^2 e_i}{m_i}. \quad (8)$$

This expression contains within-task sampling noise only because every task is included in every step.

The connection to optimization follows from the descent lemma. If F is L -smooth and $\theta' = \theta - \eta DA$, then, conditional on the current student state and allocation,

$$\mathbb{E}[F(\theta')] \leq F(\theta) - \eta \nabla F^\top D \nabla F + \frac{\eta^2 L}{2} (\|D \nabla F\|_2^2 + V(m)). \quad (9)$$

Only $V(m)$ depends on the allocation. This calculation explains both the variance objective and the use of optimizer-scaled coordinates. It omits momentum and gradient clipping.

For a fixed total of G micro-batches, minimizing $V(m)$ gives the continuous Neyman allocation

$$m_i \propto w_i \sqrt{e_i}. \quad (10)$$

We call this rule GPAS. For bounded counts, we find $z > 0$ such that

$$x_i = \text{clip}\left(\frac{w_i \sqrt{e_i}}{z}, m_{\min}, m_{\max}\right), \quad \sum_{i=1}^M x_i = G, \quad (11)$$

and apply largest-remainder rounding among coordinates not fixed at a bound. The result respects the bounds and sums to G .

4.2 WALL-CLOCK OBJECTIVE

Let τ_i be the measured marginal critical-path time of one micro-batch from task i . With one resident teacher slot and a fixed teacher order, the number of teacher visits and transfers per step does not depend on m . We model the step time as

$$t(m) = C + \sum_{i=1}^M m_i \tau_i, \quad (12)$$

where C contains the allocation-independent part of teacher loading, offloading, synchronization, and the optimizer update.

For a wall-clock budget T , the run completes approximately $T/t(m)$ steps. The scaled variance of their mean gradient estimate is therefore $V(m)t(m)/T$. The cost-aware allocation minimizes

$$J_C(m) = \left(C + \sum_{i=1}^M m_i \tau_i\right) \left(\sum_{i=1}^M \frac{w_i^2 e_i}{m_i}\right). \quad (13)$$

This derivation gives the product form directly. When $C = 0$, Cauchy–Schwarz gives

$$m_i \propto w_i \sqrt{\frac{e_i}{\tau_i}}. \quad (14)$$

For $C > 0$, there is no corresponding closed form. At $C = 0$ the optimum is Equation 14; as C becomes dominant, the objective reduces to minimizing $V(m)$ and the optimum approaches GPAS. Cost-GPAS uses the measured C and minimizes Equation 13 over the feasible integer counts. With $M = 4$, we enumerate this small set. Feasibility requires $Mm_{\min} \leq G \leq Mm_{\max}$. The first step uses the bounded uniform allocation because no noise or timing measurements are available.

Generated-token count can replace measured time when the runtime does not expose critical-path timing. It is not the default because it omits teacher transfer, waiting, and optimizer costs.

4.3 ONLINE MEASUREMENT

All statistics needed for the next allocation are obtained during the current step. For task i , define

$$\hat{e}_i = \frac{\sum_{s=1}^{m_i} \|Dg_{i,s}\|_2^2 - m_i \|Da_i\|_2^2}{m_i - 1}. \quad (15)$$

This is the unbiased sample variance summed over the optimizer-scaled coordinates. The sum contains roughly 10^9 coordinates. Under an effective independent-coordinate approximation, its relative standard error is $\sqrt{2/d_{\text{eff}}}$, even when $m_i = 2$. We therefore use the current $\hat{e}_i$ directly for the next

allocation instead of smoothing it. The experiment logs its step-to-step relative change to check this approximation.

Micro-batch times vary more because response length is heavy-tailed. We retain an exponential average for the timing model,

$$\tau_i \leftarrow \rho_\tau \tau_i + (1 - \rho_\tau) \hat{\tau}_i, \quad C \leftarrow \rho_\tau C + (1 - \rho_\tau) \hat{C}. \quad (16)$$

The timing decay is fixed from the system measurement protocol, not selected from training loss. The allocation has two tunable count constraints, $m_{\min}$ and $m_{\max}$.

We report the variance headroom realized by the current counts as

$$H = \frac{\sum_{i=1}^M w_i^2 e_i / m_i^{\text{unif}}}{\sum_{i=1}^M w_i^2 e_i / m_i}, \quad (17)$$

where m^{unif} is the bounded uniform integer allocation with the same G . A value near one means that the adaptive counts provide little variance reduction. For Cost-GPAS, H measures variance only; GPU hours measure the effect of the time objective.

Inputs: fixed weights w ; $G, c, R, m_{\min}, m_{\max}$; task prompt streams and teachers; measured timing model.

Initialize m with the bounded uniform allocation; mark e, τ , and C as unknown.

For each optimizer step:

1. Read $D = (\sqrt{v} + \epsilon)^{-1}$. If the measurements are known, set m using bounded GPAS or by minimizing Equation 13 over the feasible counts for Cost-GPAS.
2. Zero the first-moment observation A and taskwise second-moment observation U .
3. For every task i , take the next $m_i c$ prompt groups without replacement, generate R responses per group, and score them with teacher i . For each micro-batch s , form its hierarchical mean loss and gradient $g_{i,s}$; add $(w_i/m_i)g_{i,s}$ to A , $(w_i/m_i)g_{i,s}^2$ to U , $g_{i,s}/m_i$ to a_i , and $\|Dg_{i,s}\|_2^2$ to the scalar sum.
4. Set $e_i = \hat{e}_i$ using Equation 15. Update τ_i and C from the measured timing components using Equation 16.
5. Apply one AdamW update using A as the first-moment observation and U as the second-moment observation. Log m, e, τ, C, H , task losses, tokens, and timing.

IcePop truncation, ORM terms, rollout generation, and teacher scoring remain inside the existing per-micro-batch loss path.

Algorithm 1: All-task micro-batch GPAS for one MOPD training run.

5 EXPERIMENTS

The experiments separate calculation checks from the large-model efficiency test. The controlled study verifies integer allocation, gradient variance, and optimizer-state formulas. The four-teacher study tests whether those local effects reduce MOPD teacher loss per optimizer step and per GPU hour.

5.1 CONTROLLED CHECKS

We use three Gaussian micro-batch gradients in three coordinates and a fixed diagonal AdamW scaling that changes their noise ranking. Every simulated step contains $G = 16$ micro-batches with $2 \leq m_i \leq 12$; hence all three tasks are present. Bounded largest-remainder rounding gives counts (6, 5, 5) for Uniform, (11, 3, 2) for allocation by raw noise, (3, 3, 10) for GPAS, and (3, 5, 8) for the zero-overhead Cost-GPAS rule in Equation 14. Each method uses the same target weights and common random numbers over 20,000 trials.

GPAS has optimizer-scaled gradient variance 0.679 relative to Uniform, close to the calculated value 0.681. Allocation by raw noise has relative variance 2.316. The zero-overhead Cost-GPAS rule has

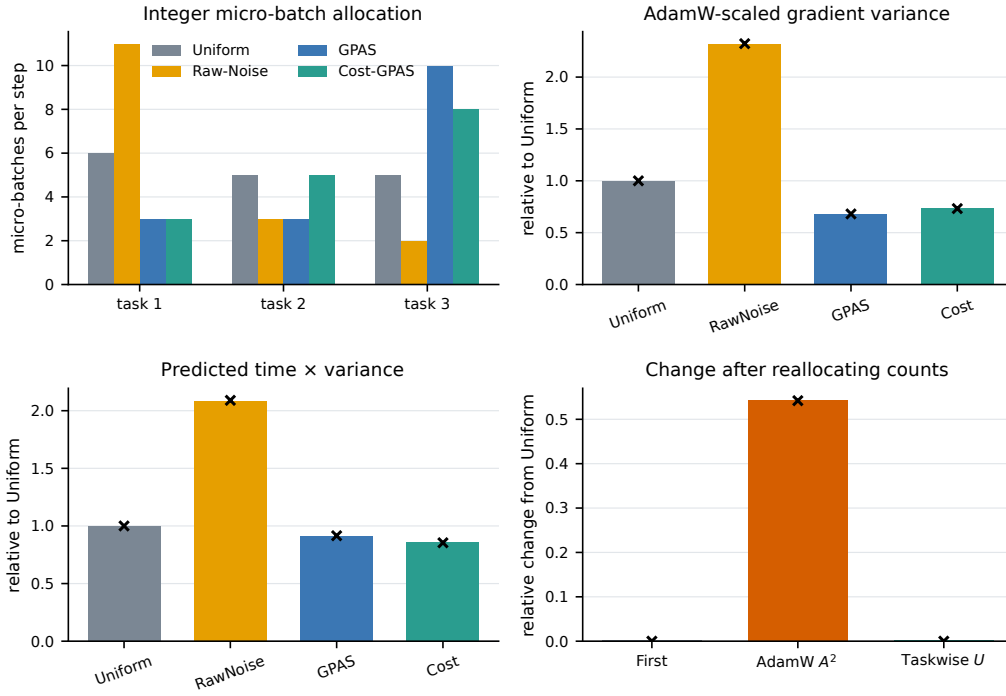


Figure 1: Controlled checks for all-task allocation. Bars are Monte Carlo estimates, and crosses are calculated values. GPAS uses optimizer-scaled within-task noise; Cost-GPAS also uses task cost. The last panel compares optimizer observations before and after reallocating integer counts; this state check is discussed in Appendix A.3.

a predicted time–variance product of 0.857, close to the calculated value 0.854. These results check the allocation formulas; they do not establish large-model training efficiency.

5.2 PREDICTIONS TESTED IN MOPD

Table 1: Empirical claims tested in the MOPD runs.

Claim	Measurement	Comparison
Adaptive counts reduce held-out variance	H and held-out variance of $\sum_i w_i a_i$	GPAS vs. bounded Uniform
The cost term reduces loss per GPU hour	Teacher loss against GPU hours and timing traces	Cost-GPAS vs. GPAS and Uniform–TW

At each selected checkpoint, every task supplies a bank of independently generated micro-batches. One half estimates e_i and the allocation; the other half measures the variance of the resulting weighted gradient, then the halves are exchanged. This separates score estimation from evaluation while using all micro-batches. Appendix A.3 describes separate checks of the sample-variance estimator and the taskwise second-moment identity.

5.3 FOUR-TEACHER SETUP

Every method initializes the student from the same Qwen3-1.7B revision and checkpoint (Qwen Team, 2025). Four fixed specialists provide teacher feedback for mathematics, code, instruction following, and science. The run manifest records the model and tokenizer revisions, precision, sharding, data versions, prompt templates, hardware, and software.

One micro-batch contains $c = 4$ prompt groups and four independently sampled responses per prompt, for 16 attempted responses. Each step processes $G = 16$ micro-batches. The bounded counts use $m_{\min} = 2$ and $m_{\max} = 8$; Uniform therefore assigns four micro-batches to every task. Reverse KL is averaged over valid tokens in each response, over the four responses for a prompt, and then over the four prompts in a micro-batch. The maximum response length is 8,192 tokens. Every attempted response is included in the reported response count and resource totals.

The main training objective is Equation 1, with the fixed inverse-initial-loss weights in Equation 2. We report both this weighted loss and the four task losses.

Every run starts from the same student and allocation state. The first update uses Uniform and initializes e_i and τ_i ; subsequent updates use the specified rule. The total budget is 128,000 attempted responses. Because G , c , and the number of responses per group are fixed, every method takes the same number of optimizer steps. The learning-rate schedule is a function of cumulative attempted responses.

Each task has a fixed stream of 16,000 training prompts. Prompts are consumed without replacement. A task assigned the maximum count for all 500 steps uses exactly $500 \times 8 \times 4 = 16,000$ prompts, so no stream cycles. The upper bound prevents one task from occupying more than half of a step and limits abrupt allocation changes. Since $m_i^{\text{unif}} = 4$ and $m_i \leq 8$, Equation 17 satisfies $H \leq 2$ for every nonnegative noise vector. We report this bound with the measured H .

5.4 ALLOCATIONS AND OPTIMIZER CONTROLS

Table 2: MOPD configurations. TW denotes the taskwise second moment in Equation 5; Conv. denotes conventional AdamW. All rows use the same w_i , G , and micro-batch size.

Allocation	Count rule	AdamW state
Uniform-Conv.	bounded Uniform	Conv.
Uniform-TW	bounded Uniform	TW
GPAS	$w_i \sqrt{e_i}$	TW
Cost-GPAS	numerical minimizer of Equation 13	TW

Uniform-Conv. is the standard-practice reference, and Uniform-TW is the method baseline. Their comparison is the only end-to-end taskwise-versus-conventional state ablation. GPAS is compared with Uniform-TW at matched optimizer steps; Cost-GPAS is compared with GPAS and Uniform-TW in GPU hours.

All configurations use a single training seed. For every task, they consume the same shuffled prompt stream in the same order; the n th requested group for a task is identical across configurations. Different allocations reach different prefix lengths at a given step. Matching the streams is a paired variance-reduction device and does not estimate variation across training seeds.

5.5 ALL-TASK EXECUTION AND COST ACCOUNTING

Every update processes all four teachers; only the number of micro-batches per teacher changes. Teacher placement, load order, and overlap policy are identical across methods. With one resident teacher slot, the four teachers are processed in a fixed cyclic order, and the final resident teacher is retained for the next update. This is the system used in the experiment.

Each task visit records load and offload time, teacher-ready time, rollout, scoring, backward, optimizer time, peak HBM, attempted responses, valid tokens, and generated tokens. Transfer tail is $\max(0, t_{\text{ready}} - t_{\text{rollout end}})$. GPU hours equal wall time times the number of reserved GPUs, including transfer and waiting. We obtain $\hat{\tau}_i$ from measured marginal critical-path time per micro-batch and $\hat{C}$ from the remaining fixed timing components. Generated tokens per micro-batch are a fallback when critical-path timing is unavailable. We report $C/(\sum_i m_i \tau_i)$ with the timing results.

The fixed-objective loss scaling remains valid with concurrent teacher slots, but the allocation-dependent step time becomes $t(m) = C + \max_i m_i \tau_i$ under ideal overlap. GPAS is unchanged. Cost-GPAS must be rederived and solved using that system’s time model; the serial Cost-GPAS rule is not transferred unchanged.

5.6 OUTCOMES AND UNCERTAINTY

The first primary plot reports weighted teacher loss against optimizer steps for GPAS and Uniform-TW. The second reports weighted teacher loss against GPU hours for Cost-GPAS, GPAS, and Uniform-TW. The main table reports final weighted and task-level loss, generated tokens and GPU hours to a common loss threshold, peak HBM, responses and tokens per GPU hour, teacher transfer time, the fixed overhead ratio, and update-time quantiles. Generated-token count is a table entry, not a primary horizontal axis. Early, middle, and late checkpoints from the Uniform-TW run provide the fixed micro-batch banks used for the mechanism test in Table 1.

Held-out loss intervals use paired prompt bootstrap with fixed evaluation generation seeds. Capability evaluation reports MATH-500 greedy pass@1, a fixed LiveCodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond average@4. For every run we log the integer counts, raw noise estimates, exponentially averaged timing estimates, H , the step-to-step relative change in $\hat{e}_i$, the fraction of steps at each bound, and task-level learning curves. The timing decay, count bounds, and common loss threshold are fixed in the run manifest before the training results are examined.

6 LIMITATIONS AND CONCLUSION

GPAS uses a local variance estimate and the current taskwise AdamW scaling. Both change during training. We use the current noise estimate without an exponential average and report its relative change between steps. The realized headroom H measures whether the resulting allocation reduces local variance.

The variance derivation assumes independent, equal-size micro-batches. The experiment uses non-cycling prompt streams to support this assumption, but generations from a shared model can still be correlated. The serial Cost-GPAS derivation also assumes the additive time model in Equation 12. Measured end-to-end GPU hours test whether that model is useful. Systems that execute teachers concurrently require a different time objective.

Taskwise AdamW is required to keep the optimizer scaling target independent of the allocation. It differs from conventional AdamW and uses one additional parameter-sized persistent buffer. The temporary task-mean buffer adds the same amount again. The Uniform-Conv. reference measures the practical effect of this optimizer change.

Inverse-initial-loss weights are unsuitable when a task has near-zero initial KL, as can occur with a self-teacher. Such tasks require an absolute KL target or another scale fixed before training; the present experiments do not include them.

The comparison fixes w_i , prompt streams, micro-batch size, and G , then changes only m_i . GPAS allocates by optimizer-scaled within-task noise. Cost-GPAS also uses the measured serial timing model. If H remains near one, uniform counts are sufficient.

When $G < 2M$, the all-task estimator is not applicable and task subsampling requires a separate method with inclusion-probability corrections.

AI USE STATEMENT

We used generative AI tools for language editing and manuscript organization. The authors reviewed every assisted revision and take responsibility for the text, claims, proofs, code, and experimental results.

ETHICS STATEMENT

The study evaluates optimization methods for existing language models with reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect capability scores. We record data provenance, invalid outputs, and verifier failures to support careful interpretation.

REPRODUCIBILITY STATEMENT

Section 5 specifies the micro-batch hierarchy, fixed objective weights, integer allocation, taskwise AdamW observation, resource accounting, baselines, and matched prompt sequences. The released artifact includes controlled-check code and seeds, the run manifest, model and tokenizer revisions, initial-loss weights, prompt streams, optimizer and allocation states, micro-batch counts, noise and timing logs, per-prompt evaluation output, and source data for every figure and table.

REFERENCES

- Guillaume Alain, Alex Lamb, Chinnadhurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL <https://arxiv.org/abs/1511.06481>.
- Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL <https://arxiv.org/abs/2312.02406>.
- Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL <https://proceedings.mlr.press/v80/chen18a.html>.
- William G. Cochran. *Sampling Techniques*. Wiley, 3 edition, 1977.
- Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.19098>.
- Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient LLM post-training, 2025. URL <https://arxiv.org/abs/2510.01135>.
- Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL <https://proceedings.mlr.press/v80/katharopoulos18a.html>.
- Vitaly Kurin, Alessandro De Palma, Ilya Kostrikov, Shimon Whiteson, and M. Pawan Kumar. In defense of the unitary scalarization for deep multi-task learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL <https://opt-ml.org/papers/2023/paper20.pdf>.
- Zhaoyi Li, Deyang Kong, Yuan Wei, Evan Yang, Ranran Shen, Mahardika Krisna Ihsani, Ming Yang, Wei Zhang, Chuan Hao, Jian Yang, Ran Tao, Bryan Dai, Shikun Zhang, Wei Ye, Ying Wei, and Defu Lian. Every coin has two sides: On the dual nature of generalization in on-policy distillation of large language models, 2026. URL <https://arxiv.org/abs/2608.16647>.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL <https://arxiv.org/abs/2110.14048>.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL <https://arxiv.org/abs/2606.30406>.
- Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learning, 2026. URL <https://arxiv.org/abs/2604.28020>.

Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4): 558–625, 1934. doi: 10.2307/2342192.

Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL <https://arxiv.org/abs/1810.04650>.

Zechen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D³-mopd: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL <https://arxiv.org/abs/2608.24987>.

Zhenting Wang, Guofeng Cui, Kun Wan, and Wentian Zhao. DUMP: Automated distribution-level curriculum learning for RL-based LLM post-training, 2025. URL <https://arxiv.org/abs/2504.09710>.

Derrick Xin, Behrooz Ghorbani, Justin Gilmer, Ankush Garg, and Orhan Firat. Do current multi-task optimization methods in deep learning even help? In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL <https://arxiv.org/abs/2001.06782>.

Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts, 2025. URL <https://arxiv.org/abs/2506.02177>.

A DERIVATIONS AND EVALUATION DETAILS

A.1 FIXED OBJECTIVE AND TASKWISE MOMENTS

For a fixed allocation m , assume the micro-batch gradients for task i are independent with mean h_i and element-wise second moment s_i . Then

$$\mathbb{E}[A] = \sum_i \frac{w_i}{m_i} \sum_{s=1}^{m_i} \mathbb{E}[g_{i,s}] = \sum_i w_i h_i. \quad (18)$$

Replacing each gradient by its element-wise square before averaging gives

$$\mathbb{E}[U] = \sum_i \frac{w_i}{m_i} \sum_{s=1}^{m_i} \mathbb{E}[g_{i,s}^2] = \sum_i w_i s_i. \quad (19)$$

Both conditional targets are independent of the integer counts. In contrast,

$$\mathbb{E}[A^2] = \left(\sum_i w_i h_i \right)^2 + \sum_i \frac{w_i^2}{m_i} \text{Var}(g_i) \quad (20)$$

coordinate by coordinate, so conventional AdamW’s observation changes with m .

A.2 GRADIENT VARIANCE AND ITS WITHIN-STEP ESTIMATE

Let D be fixed during a step. Independence across tasks and micro-batches gives

$$\mathbb{E}\|D(A - \mathbb{E}[A])\|_2^2 = \sum_i w_i^2 \mathbb{E}\|D(a_i - h_i)\|_2^2 \quad (21)$$

$$= \sum_i \frac{w_i^2}{m_i} \mathbb{E}\|D(g_i - h_i)\|_2^2, \quad (22)$$

which is Equation 8.

For one task, expand the centered sum of squares:

$$\sum_{s=1}^{m_i} \|D(g_{i,s} - a_i)\|_2^2 = \sum_{s=1}^{m_i} \|Dg_{i,s}\|_2^2 - m_i \|Da_i\|_2^2. \quad (23)$$

Dividing by $m_i - 1$ gives Equation 15, the standard unbiased sample-variance estimate summed over optimizer-scaled coordinates. The estimate is undefined at $m_i = 1$, which motivates $m_{\min} = 2$.

A.3 AUXILIARY ESTIMATOR AND OPTIMIZER-STATE CHECKS

At early, middle, and late Uniform-TW checkpoints, independently generated micro-batch banks are divided in half. Each half estimates $\hat{e}_i$ and an allocation, which are evaluated on the other half. Exchanging the halves gives held-out estimator error. We also report $|\hat{e}_i^{(t)} - \hat{e}_i^{(t-1)}|/\hat{e}_i^{(t-1)}$ from the training log to check the decision to use the current estimate without smoothing.

The controlled experiment evaluates U and A^2 under Uniform and GPAS counts. Over 200,000 simulated steps, the conventional observation changes by 0.543 in relative norm, compared with 0.542 from Equation 20. The taskwise observation changes by 0.00092 from Monte Carlo error, while Equation 6 gives zero. These are implementation checks, not empirical claims about training speed.

A.4 VARIANCE AND COST ALLOCATIONS

Write $b_i = w_i \sqrt{e_i}$. With equal cost, Cauchy–Schwarz gives

$$\left(\sum_i m_i\right) \left(\sum_i \frac{b_i^2}{m_i}\right) \geq \left(\sum_i b_i\right)^2. \quad (24)$$

For fixed G , equality holds at $m_i \propto b_i$, which is GPAS. With task-dependent cost and zero fixed overhead,

$$\left(\sum_i m_i \tau_i\right) \left(\sum_i \frac{b_i^2}{m_i}\right) \geq \left(\sum_i b_i \sqrt{\tau_i}\right)^2. \quad (25)$$

Equality holds at $m_i \propto b_i/\sqrt{\tau_i}$, which is Cost-GPAS. The term $CV(m)$ in Equation 13 removes this equality when $C > 0$, so Cost-GPAS evaluates the full objective on the feasible integer set. Reported values use the selected integer counts.

A.5 MOPD GRADIENT AND MICRO-BATCH NORMALIZATION

For a student-generated prefix z , write $q_v = \pi_\theta(v \mid z)$ and $t_v = T_i(v \mid z)$. The sampled-token reverse-KL score is

$$\text{sg} \left[\log \frac{q_V}{t_V} \right] \nabla_\theta \log q_V, \quad V \sim q. \quad (26)$$

The implementation averages tokens within a response, the R responses within a group, and the c groups within a micro-batch. IcePop truncation and ORM terms precede this mean; the factor w_i/m_i follows it.

A.6 OPTIMIZER CLOCK, LOGS, AND RESUME TESTS

All methods process the same G micro-batches and attempted responses per step and therefore use the same optimizer clock. Tests compare w_i/m_i accumulation with separately averaged task gradients and verify that the Monte Carlo mean of U is unchanged when m changes.

Checkpoints store m , the current e_i , the timing averages τ_i and C , initialization and rounding state, prompt positions, and the AdamW variant. Micro-batch logs include counts, loss, norm contributions, clipping, timing, GPU count, and peak HBM. A resume test reproduces the next allocation, moments, and parameter update.